



รายงานปฏิบัติการ PS07: Concurrency and Thread

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ภาพรวมของโปรแกรม	1
1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน	1
1.2 โครงสร้างของโปรแกรม	1
1.3 การคอมไพล์	3
2 Task 1: รันด้วย thread เดียว	3
2.1 การคอมไพล์และสภาพแวดล้อม	4
2.2 ผลการรันด้วย thread เดียว	4
3 Task 2: หลาย thread โดยไม่บังคับ context switch	4
3.1 รันบน CPU 22 ตัว	5
3.2 รันบน CPU ตัวเดียวด้วย taskset	6
3.3 ทำไม <code>balance = balance + AMOUNT</code> จึงไม่ปลอดภัย	6
3.4 เปรียบเทียบผลบน CPU 22 ตัวกับ CPU ตัวเดียว	7
4 Task 3: บังคับ context switch ระหว่างการอ่านกับการเขียน	8
4.1 การแก้โปรแกรม	8
4.2 รันบน CPU 22 ตัว	8
4.3 รันบน CPU ตัวเดียว: เทียบโหมดปกติกับโหมด <code>yield</code>	9
5 อภิปรายผลและสาเหตุของ race condition	10
5.1 สาเหตุของ race condition ในโปรแกรมนี้	10
5.2 ทำไมผลจึงไม่เหมือนกันในแต่ละครั้ง	10
5.3 บทบาทของ <code>yield</code>	11
5.4 สรุปผลการทดลองทั้งหมด	11
5.5 แนวทางแก้ไข	11
สรุปผลการทดลอง	11
เอกสารอ้างอิง	12
ภาคผนวก: ซอร์สโค้ดทั้งหมดของ ps7.c	12

สภาพแวดล้อมที่ใช้ในการทดลอง

การทดลองทั้งหมดทำบน Linux ที่ทำงานอยู่บน WSL2 ของเครื่องคอมพิวเตอร์เครื่องเดียว จุดที่มีผลต่อการทดลองนี้โดยตรงคือจำนวน CPU เพราะ thread ที่รันบน CPU คนละแกนทำงานพร้อมกันได้จริง ไม่ต้องรอให้ระบบปฏิบัติการสลับให้ รายละเอียดของสภาพแวดล้อมเป็นดังนี้

Table 1: ข้อมูลสภาพแวดล้อมของระบบที่ใช้ในการทดลอง

รายการ	รายละเอียด
ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS ทำงานบน WSL2 ของ Windows 11 Home Single Language (build 26200)
Kernel	6.6.87.2-microsoft-standard-WSL2 (x86-64, SMP PREEMPT_DYNAMIC)
CPU	Intel Core Ultra 9 185H ที่ Linux มองเห็นเป็น 22 logical CPU (11 core แกนละ 2 thread)
คอมไพเลอร์	gcc 13.3.0 ไม่ระบุระดับ optimization จึงใช้ค่าเริ่มต้นคือ -O0
Thread library	POSIX threads ของ glibc 2.39 (ลิงก์ด้วย -lpthread)
เครื่องมือเสริม	taskset (util-linux) สำหรับบังคับให้ process ทำงานบน CPU ที่กำหนด

หมายเหตุเรื่องหลักฐาน: ผลการรันทุกครั้งที่แสดงในรายงานเป็นภาพหน้าจอจริงจาก terminal ตัวเลขทุกตัวที่อ้างถึงในคำอธิบายและในตารางสรุปอ่านมาจากภาพโดยตรง ภาพทั้งหมดเก็บไว้ที่ PS07/result/screenshots/ และลำดับคำสั่งที่ใช้ถ่ายภาพอยู่ใน PS07/docs/COMMANDS.md

1 ภาพรวมของโปรแกรม

1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน

ตัวอย่างในคาบเรียนใช้ thread หลายตัวบวกค่าตัวนับ (counter) ตัวเดียวกันไปในทิศทางเดียว รายงานนี้เลือกสถานการณ์ที่ต่างออกไปคือ **บัญชีเงินฝากร่วมหนึ่งบัญชี** ที่มีตู้ ATM หลายตู้ทำรายการพร้อมกัน โดยแต่ละตู้คือ thread หนึ่งตัว และยอดเงินในบัญชีคือตัวแปรที่ทุก thread ใช้ร่วมกัน

- บัญชีเปิดด้วยยอดเงิน 1,000 บาท (INITIAL_BALANCE)
- thread ครั้งแรกเป็น **ตู้ฝากเงิน** ฝากครั้งละ 1 บาท ส่วน thread ครั้งหลังเป็น **ตู้ถอนเงิน** ถอนครั้งละ 1 บาท ทุก thread ทำรายการเท่ากันคือ tx ครั้ง
- เงินที่ไหลเข้าจึงเท่ากับเงินที่ไหลออกเสมอ **ยอดเงินสุดท้ายที่ถูกต้องต้องเท่ากับ 1,000 บาท** ไม่ว่าจะใช้กี่ thread และไม่ว่าจะทำรายการกี่ครั้ง

สถานการณ์นี้ต่างจากตัวอย่างในคาบเรียนสามข้อ ข้อแรกคือมีการเปลี่ยนค่า**สองทิศทาง** (ทั้งเพิ่มและลด) ตามที่ใบงานกำหนดให้มีทั้ง increase และ decrease ผลของ race condition จึงออกมาได้ทั้งเป็นบวกและเป็นลบ ข้อที่สองคือความผิดพลาดมีความหมายชัดเจน ถ้ายอดสุดท้ายสูงกว่า 1,000 บาทแปลว่ารายการถอนบางรายการหายไป (ธนาคารเสียเงิน) ถ้าต่ำกว่าแปลว่ารายการฝากบางรายการหายไป (ลูกค้าเสียเงิน) และข้อที่สามคือโปรแกรม**ตรวจสอบตัวเองได้** เพราะแต่ละ thread นับจำนวนรายการที่ตัวเองทำเสร็จไว้ในตัวแปร local ที่ไม่มีใครใช้ร่วม จึงพิสูจน์ได้ว่ารายการทุกรายการถูกทำจริงครบถ้วน สิ่งที่มีแต่ยอดเงินในตัวแปรที่ใช้ร่วมกันเท่านั้น

1.2 โครงสร้างของโปรแกรม

ตัวแปรที่ทุก thread ใช้ร่วมกันมีเพียงตัวเดียวคือ balance ซึ่งเป็นตัวแปร global จึงอยู่ในส่วน data ของ process และทุก thread ใน process เดียวกันมองเห็นตำแหน่งหน่วยความจำเดียวกัน ส่วนข้อมูลของแต่ละตู้เก็บใน struct atm หนึ่งชุดต่อหนึ่ง thread

```

1 #define INITIAL_BALANCE 1000L
2 #define AMOUNT          1L
3 #define MAX_THREADS     64
4
5 long balance = INITIAL_BALANCE;
6
7 int use_yield = 0;           /* set once in main() before any thread starts */
8
9 enum role { DEPOSITOR, WITHDRAWER, BOTH };
10
11 struct atm {
12     pthread_t tid;
13     enum role role;
14     long tx;                  /* transactions this thread must make */
15     long deposits;           /* written once by the thread when done: */
16     long withdrawals;        /* how many transactions it really made */
17 };

```

Code 1: ตัวแปรที่ใช้ร่วมกันและข้อมูลประจำตัวของแต่ละ thread

การฝากและถอนเขียนเป็นฟังก์ชันสั้น ๆ ที่แก้ค่า balance ตรง ๆ โดยไม่มี lock หรือกลไก synchronization ใด ๆ ป้องกันไว้ ซึ่งเป็นจุดที่โง่งานกำหนดให้มี (without proper synchronization)

```

1 void deposit_plain(void)
2 {
3     balance = balance + AMOUNT;
4 }
5
6 void withdraw_plain(void)
7 {
8     balance = balance - AMOUNT;
9 }

```

Code 2: การฝากและถอนเงินที่ไม่มีการป้องกัน

ฟังก์ชันที่ทุก thread รันคือ atm_thread() ซึ่งวนทำรายการตามบทบาทของตัวเอง ตัวนับ done_dep และ done_wdr เป็นตัวแปร local ที่อยู่บน stack ของแต่ละ thread แยกกัน จึงไม่มีทางเกิด race condition และค่าที่นับได้ถูกต้องเสมอ thread เขียนค่าเหล่านี้กลับเข้า struct atm ของตัวเองเพียงครั้งเดียวตอนทำงานเสร็จ ส่วน pointer to function สองตัวแรก (deposit และ withdraw) เลือกว่าจะใช้ฟังก์ชันฝาก-ถอนแบบใด ใน Task 1 และ Task 2 จะเป็น deposit_plain() และ withdraw_plain() เสมอ ส่วนแบบ _yield ใช้ใน Task 3

```

1 void *atm_thread(void *arg)
2 {
3     struct atm *a = arg;
4     void (*deposit)(void) = use_yield ? deposit_yield : deposit_plain;
5     void (*withdraw)(void) = use_yield ? withdraw_yield : withdraw_plain;
6     long done_dep = 0, done_wdr = 0;
7     long i;
8
9     if (a->role == DEPOSITOR || a->role == BOTH)
10         for (i = 0; i < a->tx; i++) {
11             deposit();
12             done_dep++;
13         }
14     if (a->role == WITHDRAWER || a->role == BOTH)
15         for (i = 0; i < a->tx; i++) {
16             withdraw();
17             done_wdr++;
18         }
19
20     a->deposits = done_dep;
21     a->withdrawals = done_wdr;
22     return NULL;
23 }

```

Code 3: ฟังก์ชันที่แต่ละ thread รัน

ฟังก์ชัน main() รับค่าจาก command line ในรูปแบบ ./ps7 [threads] [tx_per_thread] [yield] ตัวแรกคือจำนวน

thread ตัวที่สองคือจำนวนรายการต่อ thread (ค่าเริ่มต้นคือ 1 thread และ 1,000,000 รายการ คำสั่ง ./ps7 เหยย ๆ ตามใบงานจึงใช้ได้) และตัวที่สามซึ่งไม่บังคับคือคำว่า yield สำหรับ Task 3 ตัวแปร use_yield ถูกตั้งค่าเพียงครั้งเดียวก่อนสร้าง thread หลังจากนั้นทุก thread อ่านอย่างเดียว จึงไม่เกิด race condition กับตัวแปรนี้ จากนั้นกำหนดบทบาทของแต่ละ thread ตามตารางด้านล่าง สร้าง thread ทั้งหมดด้วย pthread_create() ก่อนแล้วจึงรอทุกตัวด้วย pthread_join() เพื่อให้ทุก thread ทำงานซ้อนกันได้ สุดท้ายรวมจำนวนรายการจากทุก thread มาคำนวณยอดที่ควรจะเป็น (expected) แล้วเทียบกับยอดจริง (actual)

Table 2: บทบาทของ thread ตามจำนวน thread ที่ระบุ

จำนวน thread	บทบาท
1	thread เดียวฝากเงิน tx ครั้งแล้วจึงถอน tx ครั้ง ใช้เป็นผลอ้างอิงแบบทำทีละรายการตามลำดับ (Task 1)
เลขคู่ตั้งแต่ 2 ถึง 64	thread ครึ่งแรกฝากเงินอย่างเดียว ครึ่งหลังถอนเงินอย่างเดียว แต่ละตัวทำ tx รายการ (Task 2 และ Task 3)
เลขคี่ที่มากกว่า 1	ไม่รับ เพราะแบ่งฝากกับถอนให้เท่ากันไม่ได้ โปรแกรมพิมพ์วิธีใช้แล้วจบการทำงาน

แต่ละรอบการรันพิมพ์ผลเพียงบรรทัดเดียว เพื่อให้ภาพหน้าจอหนึ่งภาพแสดงผลของการรันซ้ำได้หลายรอบ ความหมายของแต่ละช่องเป็นดังนี้

```
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=... diff=... RACE
```

Code 4: รูปแบบผลลัพธ์หนึ่งบรรทัดต่อการรันหนึ่งรอบ

- [plain] หรือ [yield] บอกว่ารอบนั้นใช้ฟังก์ชันฝาก-ถอนแบบใด
- threads และ tx คือจำนวน thread และจำนวนรายการต่อ thread ที่ระบุตอนรัน
- in และ out คือจำนวนรายการฝากและถอนทั้งหมดที่ thread นับได้จริง (จากตัวนับ local)
- expected คือยอดที่ควรเป็น คำนวณจาก $1000 + in - out$ ซึ่งเท่ากับ 1,000 เสมอ
- actual คือค่าใน balance หลังทุก thread ทำงานเสร็จ และ diff คือ $actual - expected$
- คำสุดท้ายเป็น ok เมื่อ diff เท่ากับศูนย์ และเป็น RACE เมื่อยอดเงินผิด

1.3 การคอมไพล์

คอมไพล์ด้วยคำสั่งตามใบงานทุกตัวอักษร คือ gcc -o ps7 ps7.c -lpthread ตัวเลือก -lpthread บอก linker ให้ลิงก์กับ POSIX thread library ส่วนการที่ไม่ระบุระดับ optimization (ซึ่งเป็น -O0) เป็นความตั้งใจ เพราะถ้าใช้ -O2 คอมไพเลอร์มีสิทธิ์มองว่าลูปที่บวก balance ทีละ 1 เป็นลำนครังมีผลเท่ากับการบวกครั้งเดียว และรวบทั้งลูปเหลือคำสั่งเดียว ช่วงเวลาที่ thread สองตัวจะชนกันจึงแทบหายไป ทั้งที่ได้ดั่งคิดอยู่เหมือนเดิม

2 Task 1: รันด้วย thread เดียว

ใบงานข้อ 2 ให้รันโปรแกรมด้วย thread เดียวก่อน เพื่อยืนยันว่าตรรกะของโปรแกรมถูกต้อง ถ้าผลที่ได้ในขั้นนี้ผิด ความผิดพลาดในขั้นต่อไปก็จะแยกไม่ออกว่าเกิดจาก race condition หรือเกิดจาก bug ธรรมดาในโปรแกรม

2.1 การคอมไพล์และสภาพแวดล้อม

```
poomipat@DESKTOP-39EEJRM: ~$ nproc; gcc --version | head -1; gcc -o ps7 ps7.c -lpthread; ls -l ps7
22
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
-rwxrwxrwx 1 poomipat poomipat 16568 Sep 27 2026 ps7
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 1: จำนวน CPU รุ่นของ gcc และการคอมไพล์ด้วยคำสั่งตามใบงาน

จากภาพ คำสั่ง `nproc` ตอบ 22 คือ Linux มองเห็น CPU 22 ตัว ซึ่งหมายความว่า thread หลายตัวของโปรแกรมนี้ได้ CPU คนละตัวและทำงานพร้อมกันได้จริง ข้อนี้จะมีผลมากใน Task 2 คอมไพเลอร์คือ gcc 13.3.0 ของ Ubuntu คำสั่ง `gcc -o ps7 ps7.c -lpthread` ไม่พิมพ์ข้อความใดออกมาเลย แปลว่าคอมไพล์ผ่านโดยไม่มีทั้ง error และ warning และ `ls -l` แสดงไฟล์ `ps7` ขนาด 16,568 byte ที่มีสิทธิ์ execute (สิทธิ์ขึ้นเป็น `rw-rwxrwx` ทุกช่อง เพราะไฟล์อยู่บน drive C ของ Windows ที่ WSL2 mount ไว้ที่ `/mnt/c`)

2.2 ผลการรันด้วย thread เดียว

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 5); do ./ps7 1 1000000; done
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 2: Task 1: รันด้วย thread เดียว 5 รอบ ได้ยอดเงินถูกต้องทุกรอบ

คำสั่ง `./ps7 1 1000000` สร้าง thread ทำงานเพียงตัวเดียว (thread main มีหน้าที่แค่สร้างแล้วรอด้วย `pthread_join()` และไม่แตะ `balance` จนกว่า thread ทำงานจะจบ) thread ตัวนี้ฝากเงิน 1,000,000 ครั้งจนยอดขึ้นไปเป็น 1,001,000 บาท แล้วจึงถอนอีก 1,000,000 ครั้งจนยอดกลับลงมา

ผลทั้ง 5 รอบในภาพเหมือนกันทุกตัวอักษร คือ `in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK` แสดงว่า

- ตรรกะของโปรแกรมถูกต้อง ทั้งการกำหนดบทบาท การวนทำรายการ ตัวนับ local และการคำนวณยอดที่ควรเป็น
- ผลลัพธ์ **deterministic** คือรันกี่ครั้งก็ได้ค่าเดิม เพราะมี thread เพียงตัวเดียวที่อ่านและเขียน `balance` การอ่าน-บวก-เขียนแต่ละครั้งจึงจบครบก่อนเริ่มครั้งถัดไปเสมอ
- แม้ระบบปฏิบัติการจะขัดจังหวะ thread ตัวนี้กลางคันเพื่อไปรันงานอื่นได้ตลอดเวลา แต่ค่าที่ค้างอยู่ใน register ถูกเก็บไว้กับ context ของ thread และถูกคืนกลับมาครบเมื่อได้ทำงานต่อ และระหว่างนั้นไม่มี thread อื่นแก้ `balance` ค่าที่เขียนกลับจึงยังถูกต้อง

ผลนี้เป็นเส้นฐานสำหรับเทียบกับขั้นต่อไป ถ้าใช้หลาย thread แล้วยอดเงินผิดไปจาก 1,000 บาท สาเหตุจึงไม่ใช่ bug ในตรรกะของโปรแกรม แต่มาจากการที่หลาย thread แก้วตัวแปรเดียวกันพร้อมกันเท่านั้น

3 Task 2: หลาย thread โดยไม่บังคับ context switch

ใบงานข้อ 3 ให้แก้โปรแกรมให้สร้าง thread มากกว่าหนึ่งตัว และข้อ 4 ให้รันโดยไม่บังคับ context switch หลาย ๆ ครั้งเพื่อดูว่าผลไม่แน่นอน โปรแกรมนี้รับจำนวน thread จาก command line อยู่แล้ว การแก้ไขขั้นนี้จึงเป็นการสั่ง `./ps7 4 1000000` แทน `./ps7 1 1000000` ซึ่งทำให้ `main()` สร้าง thread 4 ตัว เป็นตู้ฝากเงิน 2 ตัวและตู้ถอนเงิน 2 ตัว แต่ละตัวทำ 1,000,000 รายการ รวมเป็นฝาก 2,000,000 ครั้งและถอน 2,000,000 ครั้ง ยอดที่ถูกต้องจึงยังเป็น 1,000 บาทเหมือนเดิม ตัวฟังก์ชัน `deposit_plain()` และ `withdraw_plain()` ไม่ได้แก้เลยแม้แต่บรรทัดเดียว

การทดลองนี้รันสองแบบ แบบแรกปล่อยให้ระบบปฏิบัติการจัดวาง thread บน CPU ทั้ง 22 ตัวตามปกติ แบบที่สองใช้ taskset -c 0 บังคับให้ทุก thread อยู่บน CPU หมายเลข 0 ตัวเดียว เพื่อแยกให้เห็นว่าผลที่เกิดจากกลไกใด

3.1 รันบน CPU 22 ตัว

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 10); do ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-33229 diff=-33229 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-130619 diff=-131619 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=152202 diff=+151202 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=121915 diff=+120915 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=140707 diff=+139707 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-6078 diff=-7078 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-113785 diff=-114785 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-725602 diff=-726602 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=380062 diff=+379062 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-186190 diff=-187190 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 3: Task 2: 4 thread บน CPU 22 ตัว รัน 10 รอบ ผิดทุกรอบและไม่ซ้ำกันเลย

ทั้ง 10 รอบในภาพลงท้ายด้วย RACE ค่า diff ของแต่ละรอบเรียงตามลำดับคือ -33,229, -131,619, +151,202, +120,915, +139,707, -7,078, -114,785, -726,602, +379,062 และ -187,190 ข้อสังเกตจากภาพมีดังนี้

- ผลไม่ซ้ำกันเลยสักรอบ ทั้งที่เป็นโปรแกรมเดียวกัน รับ argument เดียวกัน และรันบนเครื่องเดียวกันติดต่อกัน นี่คือพฤติกรรมแบบ non-deterministic ที่ในงานต้องการให้แสดง
- ผิดได้ทั้งสองทิศทาง มีรอบที่ยอดเกิน 4 รอบ (รายการถอนหายไปมากกว่ารายการฝาก ธนาคารเสียเงิน) และรอบที่ยอดขาด 6 รอบ (รายการฝากหายไปมากกว่า ลูกค้าเสียเงิน) รอบที่ผิดมากที่สุดยอดเงินขาดไป 726,602 บาท และรอบที่ใกล้เคียงที่สุดก็ยังผิด 7,078 บาท
- รายการครบทุกรายการ ช่อง in=2000000 out=2000000 เท่ากันทุกบรรทัด แปลว่าทุก thread ทำรายการครบตามจำนวนจริง สิ่งที่ยหายไปคือผลของการเขียนลงตัวแปร balance ไม่ใช่ตัวรายการ
- ค่า diff เป็นเพียงผลสุทธิระหว่างรายการฝากที่หายกับการถอนที่หาย ซึ่งหักล้างกันได้ จำนวนรายการที่หายไปจริงจึงมากกว่าหรือเท่ากับค่าสัมบูรณ์ของ diff เสมอ

สาเหตุที่ผิดทุกรอบในกรณีนี้คือเครื่องมี CPU มากพอให้ thread ทั้ง 4 ตัวได้ CPU คนละตัว thread จึงทำงานพร้อมกันจริง (parallelism) ไม่ต้องรอให้ระบบปฏิบัติการสลับให้ ในทุก ๆ รอบของลูป CPU สองตัวมีโอกาสอ่านค่า balance ค่าเดียวกันไปพร้อมกัน แล้วต่างคนต่างเขียนผลของตัวเองกลับ ผลของตัวที่เขียนก่อนจึงถูกเขียนทับหายไป เมื่อลูปวนเป็นล้านรอบ การชนแบบนี้จึงเกิดขึ้นเป็นจำนวนมหาศาล และจำนวนที่ชนในแต่ละรอบการรันขึ้นอยู่กับจังหวะเวลาที่ควบคุมไม่ได้ เช่น thread ไหนเริ่มก่อน ได้ CPU ตัวใด และ CPU ตัวอื่นกำลังทำอะไรอยู่

3.2 รันบน CPU ตัวเดียวด้วย taskset

```

poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 20); do taskset -c 0 ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=2001000 diff=+200000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+100000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+100000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-628150 diff=-629150 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+100000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+100000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+100000 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$

```

Figure 4: Task 2: 4 thread บน CPU ตัวเดียว รัน 20 รอบ ถูก 13 รอบ ผิด 7 รอบ

เมื่อบังคับให้ทุก thread อยู่บน CPU ตัวเดียว ในแต่ละขณะจะมี thread ทำงานได้เพียงตัวเดียว thread จะสลับกันก็ต่อเมื่อ kernel ขัดจังหวะ thread ที่กำลังทำงาน (timer interrupt เมื่อหมด time slice) แล้วเลือก thread อื่นขึ้นมาทำงานแทน ผลในภาพต่างจากกรณี CPU 22 ตัวอย่างชัดเจน

- ถูก 13 รอบ ผิด 7 รอบ รอบที่ผิดคือรอบที่ 3, 6, 8, 12, 17, 19 และ 20 ส่วนที่เหลือได้ actual=1000 diff=+0 OK
- เมื่อผิดจะผิดเป็นก้อนใหญ่ที่ลงตัว รอบที่ 6, 8, 17, 19 และ 20 ได้ diff=+1000000 พอดี ซึ่งเท่ากับงานทั้งหมดของ thread ถอนเงินหนึ่งตัว รอบที่ 3 ได้ diff=+2000000 เท่ากับงานของ thread ถอนเงินทั้งสองตัว และรอบที่ 12 ได้ diff=-629150 คือรายการฝาก 629,150 รายการหายไป

รูปแบบนี้อธิบายได้ด้วยจังหวะของ context switch ลูปของ thread หนึ่งตัว (1,000,000 รอบ) ใช้เวลาเพียงไม่กี่มิลลิวินาที thread ส่วนใหญ่จึงทำงานจบได้โดยไม่ต้องมี time slice และถึงจะโดนขัดจังหวะ จุดที่โดนขัดก็มักไม่ตกอยู่ระหว่างการอ่านกับการเขียน balance รอบส่วนใหญ่จึงถูกต้อง แต่ถ้าบังเอิญ thread ฝากเงินตัวหนึ่งโดนขัดจังหวะ**หลังจากอ่านค่า balance เข้า register แล้วแต่ยังไม่ได้เขียนกลับ** ค่าใน register นั้นจะถูกเก็บไว้กับ context ของมัน ระหว่างที่รอ thread ถอนเงินได้ขึ้นมาทำงานและถอนครบทั้ง 1,000,000 ครั้ง เมื่อ thread ฝากเงินได้กลับมาทำงานต่อ มันเขียนค่าเก่าที่ค้างอยู่ทบลงไป รายการถอนทั้งล้านรายการจึงหายไปในการเขียนครั้งเดียว ได้ diff=+1000000 พอดี กรณี diff=-629150 ก็เป็นกลไกเดียวกันในทิศตรงข้าม คือ thread ถอนเงินถือค่าเก่าค้างไว้ขณะที่ thread ฝากเงินทำไปได้ 629,150 รายการ

นี่คือ **untimely context switch** ตามที่ใบงานอธิบาย ซึ่งเกิดขึ้นได้เองตามธรรมชาติแต่ไม่บ่อย จึงต้องรันหลายครั้งถึงจะเห็น และเป็นเหตุผลที่ Task 3 ต้องบังคับให้เกิด

3.3 ทำไม `balance = balance + AMOUNT` จึงไม่ปลอดภัย

คำสั่ง C หนึ่งบรรทัดดูเหมือนทำงานเสร็จในขั้นเดียว แต่ CPU ไม่ได้บอกค่าในหน่วยความจำโดยตรง ภาพด้านล่างใช้ objdump แปลงไฟล์ ps7 ตัวเดียวกับที่ใช้รันใน t03 และ t04 กลับเป็นภาษา assembly (รูปแบบ Intel ที่เขียนปลายทางไว้ก่อนต้นทาง)

```

poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ objdump -d --no-show-raw-insn -M intel ps7 | awk '/<deposit|
withdraw>.plain>:;ret/'
000000000001269 <deposit_plain>:
1269:      endbr64
126d:      push    rbp
126e:      mov     rbp, rsp
1271:      mov     rax, QWORD PTR [rip+0x2d98]      # 4010 <balance>
1278:      add     rax, 0x1
127c:      mov     QWORD PTR [rip+0x2d8d], rax      # 4010 <balance>
1283:      nop
1284:      pop     rbp
1285:      ret
000000000001286 <withdraw_plain>:
1286:      endbr64
128a:      push    rbp
128b:      mov     rbp, rsp
128e:      mov     rax, QWORD PTR [rip+0x2d7b]      # 4010 <balance>
1295:      sub     rax, 0x1
1299:      mov     QWORD PTR [rip+0x2d70], rax      # 4010 <balance>
12a0:      nop
12a1:      pop     rbp
12a2:      ret
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$

```

Figure 5: คำสั่งเครื่องของ deposit_plain() และ withdraw_plain()

ทั้งสองฟังก์ชันมีโครงสร้างเหมือนกัน คำสั่งที่เหลือนั้นเป็นเพียงการเข้าและออกจากฟังก์ชัน ส่วนที่เป็นหัวใจของ race condition คือสามคำสั่งตรงกลาง ซึ่ง objdump ระบุไว้ท้ายบรรทัดว่าอ้างอิงตัวแปร <balance> ที่ตำแหน่ง 4010 เดียวกัน

Table 3: สามขั้นตอนของการแก้ค่า balance ที่อ่านได้จากภาพ

ขั้น	ที่อยู่	คำสั่ง	ความหมาย
Load	1271 / 128e	mov rax,QWORD PTR [...]	อ่านค่า balance จากหน่วยความจำมาไว้ใน register rax
Modify	1278 / 1295	add rax,0x1 / sub rax,0x1	บวกหรือลบ 1 เฉพาะใน register หน่วยความจำยังเป็นค่าเดิม
Store	127c / 1299	mov QWORD PTR [...],rax	เขียนค่าใน rax กลับลง balance ทับค่าที่อยู่ในหน่วยความจำขณะนั้น

ปัญหาอยู่ที่ช่วงเวลาระหว่าง Load กับ Store ถ้ามี thread อื่นแก้ balance ในช่วงนี้ ไม่ว่าจะเพราะทำงานพร้อมกันบน CPU อีกตัว (t03) หรือเพราะ thread นี้ถูกสลับออกไปกลางคัน (t04) ขั้น Store จะเขียนค่าที่คำนวณจากข้อมูลเก่าทับลงไป และการแก้ของ thread อื่นจะหายไปโดยไม่มีสัญญาณเตือนใด ๆ การอ่าน-แก้-เขียน (read-modify-write) สามขั้นนี้จึงไม่เป็น **atomic** และโค้ดส่วนนี้คือ critical section ที่ไม่มีการป้องกัน

3.4 เปรียบเทียบผลบน CPU 22 ตัวกับ CPU ตัวเดียว

Table 4: เปรียบเทียบผลของ Task 2 ทั้งสองแบบ (อ่านจากภาพ t03 และ t04)

ประเด็น	CPU 22 ตัว (t03)	CPU ตัวเดียว (t04)
จำนวนรอบที่ผิด	10 จาก 10 รอบ	7 จาก 20 รอบ
ลักษณะของค่าที่ผิด	ไม่ซ้ำกันเลย มีทั้งบวกและลบ ตั้งแต่ -726,602 ถึง +379,062	ลงตัวเป็นก้อนได้แก่ +1,000,000 จำนวน 5 รอบ, +2,000,000 และ -629,150 อย่างละ 1 รอบ
ลักษณะการทำงาน	พร้อมกันจริงบน CPU คนละตัว	ทีละตัว สลับกันเมื่อ kernel ชัดจังหวะ
การชนกันเมื่อใด	ได้ทุกรอบของ loop เมื่อ CPU สองตัวเข้าช่วง Load-Store พร้อมกัน	เฉพาะเมื่อ context switch ตกกระทหว่าง Load กับ Store พอดี

ทั้งสองแบบใช้โปรแกรมเดียวกันและผิดด้วยสาเหตุเดียวกัน คือการอ่าน-แก้-เขียนตัวแปรที่ใช้ร่วมกันโดยไม่มีการป้องกัน ต่างกันเพียงว่าอะไรเป็นตัวแทรกเข้ามาในช่วง Load-Store บนเครื่องที่มี CPU หลายตัว การทำงานพร้อมกันจริงทำให้ race condition ปรากฏแทบทุกครั้ง ส่วนบน CPU ตัวเดียว race condition จะปรากฏเฉพาะเมื่อ context switch เกิดขึ้นผิดจังหวะ ซึ่งเป็นเหตุการณ์ที่สุ่มและไม่บ่อย โปรแกรมที่มี bug แบบนี้จึงอาจผ่านการทดสอบหลายครั้งแล้วค่อยพังในภายหลัง

4 Task 3: บังคับ context switch ระหว่างการอ่านกับการเขียน

4.1 การแก้ไขโปรแกรม

ใบงานข้อ 5 ให้เขียนโปรแกรมที่บังคับให้เกิด context switch ขณะกำลังแก้ค่าตัวแปร โดยแยกคำสั่ง `counter = counter + 1` ออกเป็นการอ่านค่าเข้า register การบวก และการเขียนกลับ แล้วเรียก `yield()` ก่อนการเขียน โปรแกรมนี้จึงเพิ่มฟังก์ชันฝาก-ถอนอีกคู่หนึ่งที่เขียนสามขั้นนี้แยกกันให้เห็นชัดตามตัวอย่างในใบงาน

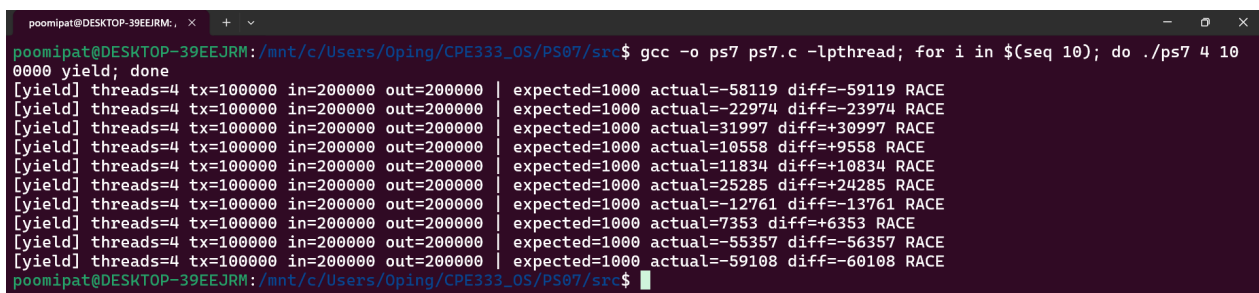
```
1 void deposit_yield(void)
2 {
3     long reg = balance;          /* Load */
4     reg = reg + AMOUNT;          /* modify */
5     sched_yield();               /* force a context switch */
6     balance = reg;              /* store */
7 }
8
9 void withdraw_yield(void)
10 {
11     long reg = balance;
12     reg = reg - AMOUNT;
13     sched_yield();
14     balance = reg;
15 }
```

Code 5: การฝากและถอนเงินที่บังคับ context switch ระหว่าง load กับ store

ตัวแปร local `reg` ทำหน้าที่แทน register ในตัวอย่างของใบงาน ค่าที่อ่านมาจึงค้างอยู่กับ thread นั้นในช่วงที่มันไม่ได้ทำงาน ส่วนฟังก์ชันที่ใช้บังคับสลับคือ `sched_yield()` จาก `<sched.h>` ซึ่งเป็น system call มาตรฐานของ POSIX ที่บอก kernel ว่า thread นี้ยอมสละ CPU และขอไปต่อท้ายคิวของ thread ที่พร้อมทำงาน ไม่ได้ใช้ `pthread_yield()` เพราะ glibc ตั้งแต่รุ่น 2.34 ประกาศเลิกใช้ฟังก์ชันนี้แล้ว และคอมไพเลอร์จะเตือนทุกครั้งที่เราเรียก

โหมคนี่เปิดด้วย argument ตัวที่สาม `yield` ซึ่งตั้งค่า `use_yield` ให้ `atm_thread()` เลือกใช้ฟังก์ชันนี้แทนคู่เดิม ส่วนอื่นของโปรแกรมเหมือนเดิมทุกอย่าง จำนวนรายการต่อ thread ลดเหลือ 100,000 เพราะ `sched_yield()` เป็น system call ที่ต้องเข้า kernel ทุกรอบของลูป จึงช้ากว่าการบวกเลขใน register หลายร้อยเท่า

4.2 รันบน CPU 22 ตัว



```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ gcc -o ps7 ps7.c -lpthread; for i in $(seq 10); do ./ps7 4 10 0000 yield; done
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-58119 diff=-59119 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-23974 diff=-23974 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=30997 diff=+30997 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=10558 diff=+9558 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=11834 diff=+10834 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=25285 diff=+24285 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-12761 diff=-13761 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=7353 diff=+6353 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-55357 diff=-56357 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-59108 diff=-60108 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 6: Task 3: คอมไพล์ใหม่แล้วรันโหมต yield บน CPU 22 ตัว 10 รอบ ผิดทุกครั้ง

คอมไพล์ใหม่ด้วยคำสั่งเดิมผ่านโดยไม่มีข้อความใด ๆ จากนั้นรันโหมต `yield` 10 รอบ ทุกบรรทัดขึ้นต้นด้วย `[yield]` และลงท้ายด้วย `RACE` ค่า `diff` ของแต่ละรอบเรียงตามลำดับคือ -59,119, -23,974, +30,997, +9,558, +10,834, +24,285, -13,761, +6,353, -56,357 และ -60,108 เป็นบวก 5 รอบและลบ 5 รอบ ไม่ซ้ำกันเลยสักรอบ รอบที่ใกล้เคียงที่สุดก็ยังมี 6,353 บาท ขณะที่ `in=200000 out=200000` เท่ากันทุกบรรทัดเช่นเคย

บนเครื่องที่มี CPU มากกว่าจำนวน thread แต่ละ thread มักได้ CPU ของตัวเอง เมื่อเรียก `sched_yield()` แล้วไม่มี thread อื่นรออยู่บน CPU นั้น kernel จะให้ thread เดิมทำงานต่อทันทีโดยไม่สลับจริง (ตามที่ `sched_yield(2)` ระบุ) ผลของ `sched_yield()` ในกรณีนี้จึงเป็นการยืดช่วงเวลาระหว่าง load กับ store ให้ยาวขึ้นเป็นระยะเวลาของ system call หนึ่งครั้ง thread บน CPU อื่นจึงมีเวลาเขียน `balance` แทรกเข้ามาในช่วงนั้นได้มาก ผลจึงผิดทุก

อบเหมือน Task 2 แต่ race condition บน CPU หลายตัวเกิดขึ้นได้อยู่แล้วแม้ไม่มี yield ภาพนี้จึงยังไม่ได้แยกให้เห็นผลของการบังคับสลับโดยตรง การทดลองถัดไปทำหน้าทีนั้น

4.3 รันบน CPU ตัวเดียว: เทียบโหมดปกติกับโหมด yield

การทดลองนี้เป็นการทดลองแบบควบคุม คือรันบน CPU หมายเลข 0 ตัวเดียวด้วย taskset -c 0 ใช้ 4 thread และ 100,000 รายการต่อ thread เท่ากันทุกรอบ รอบที่ 1-10 ใช้โหมดปกติ รอบที่ 11-20 ใช้โหมด yield สิ่งเดียวที่ต่างกันคือมีการเรียก sched_yield() ระหว่าง load กับ store หรือไม่

```

poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_05/PS07/src$ for i in $(seq 10); do taskset -c 0 ./ps7 4 100000 yield; done
i in $(seq 10); do taskset -c 0 ./ps7 4 100000 yield; done
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98985 diff=-99985 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98951 diff=-99951 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98998 diff=-99998 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98990 diff=-99990 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-99000 diff=-100000 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_05/PS07/src$

```

Figure 7: Task 3: CPU ตัวเดียว 100,000 รายการต่อ thread โหมดปกติถูกทั้ง 10 รอบ โหมด yield ผิดทั้ง 10 รอบ

- โหมดปกติ 10 รอบแรกถูกทุกรอบ (actual=1000 diff=+0 OK) เพราะลูป 100,000 รอบของแต่ละ thread สั้นมาก thread จึงมักทำงานจบก่อนหมด time slice และไม่ถูกขัดจังหวะระหว่าง load กับ store เลย
- โหมด yield 10 รอบหลังผิดทุกรอบ ค่า diff คือ -99,985, +100,000, +100,000, -99,951, +100,000, -99,998, -99,990, +100,000, +100,000 และ -100,000 ได้ +100000 พอดี 5 รอบ และอีก 5 รอบอยู่ใกล้ -100000 มาก

ผลนี้ยืนยันว่า context switch ที่เกิดขัดจังหวะคือตัวที่ทำให้เกิดการเขียนทับ ในเมื่อโปรแกรมเดียวกันบน CPU ตัวเดียวกันถูกต้องทุกครั้งเมื่อไม่มี yield แต่ผิดทุกครั้งเมื่อบังคับให้สลับระหว่าง load กับ store

ส่วนเหตุที่ค่าผิดออกมาใกล้ +100,000 หรือ -100,000 อธิบายได้ดังนี้ บน CPU ตัวเดียวที่มี 4 thread พร้อมทำงาน ทุกครั้งที่ thread เรียก sched_yield() kernel จะสลับไปยัง thread ถัดไปในคิวจริง ๆ thread ทั้ง 4 ตัวจึงผลัดกันทำงานทีละหนึ่งรายการ ในแต่ละรอบ thread หนึ่งตัวจะ store ค่าที่ค้างไว้จากรอบก่อน แล้ว load ค่าที่ตัวเองเพิ่งเขียนนั้นกลับมาทันทีก่อนจะ yield อีกครั้ง ผลคือ thread ฝากเงินทั้งสองตัวเห็นแต่ค่าที่ฝั่งฝากเขียนไว้ และเดินขึ้นไปทีละ 1 ต่อรอบเหมือนเป็นตัวนับตัวเดียว (ฝาก 200,000 ครั้งแต่มีผลแค่ 100,000) ขณะที่ thread ถอนเงินทั้งสองตัวก็เดินลงทีละ 1 ต่อรอบในแบบเดียวกัน สองฝั่งนี้ต่างเขียนทับกันตลอดเวลา และยอดสุดท้ายคือค่าของฝั่งที่เขียนเป็นคนสุดท้าย ได้แก่ $1,000 + 100,000 = 101,000$ ถ้าฝั่งฝากเขียนทีหลัง หรือ $1,000 - 100,000 = -99,000$ ถ้าฝั่งถอนเขียนทีหลัง ตรงกับ actual=101000 และ actual=-99000 ในภาพ ส่วนค่าที่คลาดจาก 100,000 ไปเล็กน้อย เช่น -99,951 เกิดจากช่วงที่ลำดับการสลับไม่ได้เป็นระเบียบพอดี เช่น ตอนที่ thread เพิ่งถูกสร้างหรือเพิ่งทำงานจบ ฝั่งใดจะได้เขียนเป็นคนสุดท้ายขึ้นอยู่กับลำดับที่ scheduler จัดให้ในรอบนั้น ผลจึงสลับบวกและลบไปมาโดยคาดเดาไม่ได้

5 อภิปรายผลและสาเหตุของ race condition

5.1 สาเหตุของ race condition ในโปรแกรมนี้

race condition คือสถานการณ์ที่ผลลัพธ์ของโปรแกรมขึ้นอยู่กับลำดับหรือจังหวะเวลาที่ thread หลายตัวเข้าถึงข้อมูลเดียวกัน ในโปรแกรมนี้มีเงื่อนไขครบทั้งสามข้อที่ทำให้เกิด race condition

1. **มีข้อมูลที่ใช้ร่วมกัน** ตัวแปร global balance อยู่ในหน่วยความจำส่วนเดียวที่ทุก thread ของ process มองเห็น ต่างจาก done_dep และ done_wdr ที่อยู่บน stack ของแต่ละ thread แยกกัน ตัวนับเหล่านี้จึงถูกต้องเสมอ (in และ out ถูกทุกบรรทัดในทุกภาพ)
2. **การแก้ค่าไม่เป็น atomic** คำสั่ง $balance = balance + AMOUNT$ ถูกแปลเป็นสามคำสั่งเครื่องคือ load, add และ store (ภาพ t05) ระหว่างสามขั้นนี้ thread อื่นแทรกเข้ามาได้เสมอ
3. **ไม่มีการกันไม่ให้เข้า critical section พร้อมกัน** (mutual exclusion) ไม่มี lock หรือกลไกใด ๆ บังคับให้ thread ทำสามขั้นนี้จบก่อนที่ thread อื่นจะเริ่ม

ผลที่ตามมาเรียกว่า **lost update** คือการแก้ค่าของ thread หนึ่งถูกเขียนทับด้วยค่าที่อีก thread หนึ่งคำนวณจากข้อมูลเก่า ตารางด้านล่างแสดงลำดับเหตุการณ์ที่ทำให้รายการถอนเงินหนึ่งรายการหายไป ขั้นที่ 3–5 จะเกิดจาก thread B ทำงานพร้อมกันบน CPU อีกตัว (t03) หรือเกิดจาก thread A ถูกสลับออกหลังขั้นที่ 2 (t04 และ t07) ก็ได้ผลเหมือนกัน

Table 5: ลำดับเหตุการณ์ที่ทำให้รายการถอนเงินหายไปหนึ่งรายการ

ลำดับ	thread A (ฝาก)	thread B (ถอน)	balance ในหน่วยความจำ
1	load: rax = 1000		1000
2	add: rax = 1001		1000
3		load: rax = 1000	1000
4		sub: rax = 999	1000
5		store: balance = 999	999
6	store: balance = 1001		1001

ฝากหนึ่งครั้งและถอนหนึ่งครั้ง ยอดที่ถูกต้องต้องกลับมาเป็น 1,000 บาท แต่ได้ 1,001 บาทเพราะ thread A เขียนค่า 1001 ซึ่งคำนวณจากค่า 1000 ที่อ่านไว้ก่อน B จะถอน ทับผลของ B ไป ถ้าขั้นที่ 5 กับขั้นที่ 6 สลับลำดับกัน ยอดจะเป็น 999 บาทแทน คือรายการฝากของ A หายไป **ผลลัพธ์ขึ้นกับว่า thread ใด store เป็นคนสุดท้าย** ซึ่งโปรแกรมไม่ได้เป็นผู้กำหนด

5.2 ทำไมผลจึงไม่เหมือนกันในแต่ละครั้ง

ลำดับการสลับของ thread ถูกกำหนดโดย scheduler ของระบบปฏิบัติการ และขึ้นกับปัจจัยที่โปรแกรมควบคุมไม่ได้ เช่น จังหวะของ timer interrupt, thread แต่ละตัวถูกสร้างเสร็จเร็วช้าต่างกันเท่าไร, ได้ CPU ตัวใด และ process อื่นในเครื่องกำลังใช้ CPU อยู่เท่าใด เมื่อลูปวนเป็นแสนถึงล้านรอบ จำนวนรูปแบบการสลับที่เป็นไปได้มีมากจนแทบไม่มีทางซ้ำกัน ผลที่ได้จึงเปลี่ยนไปทุกครั้งที่รัน ทั้งที่ใช้โปรแกรมและ input เดียวกัน นี่คือความหมายของ non-deterministic ที่เห็นใน t03, t04, t06 และ t07

ทิศทางของความผิดพลาดบอกได้ว่ารายการประเภทใดหายไปมากกว่า ถ้ารายการฝากถูกเขียนทับมากกว่า ยอดสุดท้ายจะต่ำกว่า 1,000 บาท ถ้ารายการถอนถูกเขียนทับมากกว่า ยอดจะสูงกว่า เนื่องจากทั้งสองอย่างเกิดขึ้นพร้อมกันและหักล้างกันได้ ค่า diff จึงเป็นเพียงผลสุทธิ จำนวนรายการที่หายไปจริงมีมากกว่านั้น

5.3 บทบาทของ yield

`sched_yield()` ไม่ได้สร้าง bug ขึ้นมาใหม่ bug มีอยู่แล้วตั้งแต่ Task 2 เพราะช่วงระหว่าง load กับ store เป็นช่วงที่ไม่มีการป้องกัน สิ่งที่ `sched_yield()` ทำคือทำให้ช่วงอันตรายนั้นถูกแทรกแน่นอนทุกครั้ง แทนที่จะรอให้ timer interrupt ตกลงตรงช่วงนั้นเองโดยบังเอิญ บน CPU ตัวเดียวผลจึงเปลี่ยนจากถูกถูกรอบ (t07 โหมตปกติ) เป็นผิดถูกรอบ (t07 โหมต yield) ส่วนบน CPU หลายตัวซึ่งมี race condition จากการทำงานพร้อมกันจริงอยู่แล้ว yield ทำให้ช่วงระหว่าง load กับ store ยาวขึ้น และผลก็ยิ่งผิดถูกรอบ (t06)

5.4 สรุปผลการทดลองทั้งหมด

Table 6: สรุปผลการรันทุกชุด (ตัวเลขทั้งหมดอ่านจากภาพหน้าจอ)

ภาพ	โหมต	thread/CPU	tx/thread	รอบ	รอบที่ผิด	diff ที่พบ
t02	plain	1 / 22	1,000,000	5	0	+0 ทุกรอบ
t03	plain	4 / 22	1,000,000	10	10	-726,602 ถึง +379,062 ไม่ซ้ำกัน
t04	plain	4 / 1	1,000,000	20	7	+1,000,000 (5 รอบ), +2,000,000 และ -629,150
t06	yield	4 / 22	100,000	10	10	-60,108 ถึง +30,997 ไม่ซ้ำกัน
t07	plain	4 / 1	100,000	10	0	+0 ทุกรอบ
t07	yield	4 / 1	100,000	10	10	+100,000 (5 รอบ) และ ใกล้ -100,000 (5 รอบ)

5.5 แนวทางแก้ไข

ใบงานไม่ได้ให้แก้ race condition แต่เพื่อให้เห็นว่าปัญหายู่ตรงไหน วิธีแก้ที่ตรงจุดคือทำให้การอ่าน-แก้-เขียนทั้งสามขั้นกลายเป็นหน่วยเดียวที่ถูกขัดไม่ได้ ทำได้สองแนวทาง แนวทางแรกคือครอบ critical section ด้วย mutex ให้ thread เข้าได้ทีละตัว แนวทางที่สองคือใช้คำสั่ง atomic ของ CPU ซึ่งทำ load-add-store จบในคำสั่งเดียว (บน x86-64 คือคำสั่ง `lock add`) ตัวอย่างด้านล่างเป็นเพียงแนวทาง ไม่ได้นำไปทดลองในรายงานนี้

```

1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 void deposit_mutex(void)
4 {
5     pthread_mutex_lock(&lock);
6     balance = balance + AMOUNT;
7     pthread_mutex_unlock(&lock);
8 }
9
10 void deposit_atomic(void)
11 {
12     __atomic_fetch_add(&balance, AMOUNT, __ATOMIC_SEQ_CST);
13 }

```

Code 6: แนวทางแก้ไขสองแบบ (ไม่ได้ใช้ในการทดลอง)

สรุปผลการทดลอง

โปรแกรมบัญชีเงินฝากร่วมที่ใช้ในรายงานนี้มีผู้ฝากและถอนจำนวนเท่ากัน ยอดที่ถูกต้องจึงต้องกลับมาเป็น 1,000 บาทเสมอ เมื่อรันด้วย thread เดียว (Task 1) ผลถูกต้องและเหมือนกันทุกครั้ง ยืนยันว่าตรรกะของโปรแกรมไม่มีข้อผิดพลาด

เมื่อเพิ่มเป็น 4 thread โดยไม่บังคับ context switch (Task 2) บน CPU 22 ตัวผลผิดถูกรอบและไม่ซ้ำกันเลย เพราะ thread ทำงานพร้อมกันจริงและชนกันได้ในทุกกรอบของลูป ส่วนบน CPU ตัวเดียว ผลถูกเป็นส่วนใหญ่และผิด

เป็นบางครั้ง เพราะการชนกันเกิดเฉพาะเมื่อ context switch ต่กระหว่าง load กับ store พอดี และเมื่อผิดก็ผิดทีละมาก เพราะ thread ที่ถือค่าเก่าไว้จะเขียนทับงานของ thread อื่นทั้งหมดที่ทำได้ไประหว่างนั้น

เมื่อบังคับ context switch ด้วย sched_yield() ระหว่าง load กับ store (Task 3) ผลผิดทุกอย่างทั้งบน CPU หลายตัวและ CPU ตัวเดียว และการทดลองแบบควบคุมบน CPU ตัวเดียวแสดงชัดว่า ที่จำนวนรายการเท่ากัน โหมดปกติถูกทั้ง 10 รอบ แต่โหมด yield ผิดทั้ง 10 รอบ

สาเหตุของทุกกรณีคือสิ่งเดียวกัน ได้แก่การอ่าน-แก้-เขียนตัวแปรที่ใช้ร่วมกันโดยไม่เป็น atomic และไม่มี mutual exclusion บทเรียนสำคัญคือ race condition อาจไม่ปรากฏในการทดสอบหลายครั้ง (เช่น 13 จาก 20 รอบใน t04 และ 10 รอบแรกใน t07 ที่ถูกต้อง) การรันแล้วได้ผลถูกต้องจึงไม่ได้พิสูจน์ว่าโปรแกรมที่ใช้ข้อมูลร่วมกันระหว่าง thread นั้นถูกต้อง ต้องป้องกัน critical section ด้วยกลไก synchronization ตั้งแต่ตอนออกแบบ

เอกสารอ้างอิง

1. Arpaci-Dusseau, R. H., Arpaci-Dusseau, A. C., *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books (บทที่ 26 Concurrency: An Introduction และบทที่ 27 Interlude: Thread API)
2. man 3 pthread_create, man 3 pthread_join และ man 2 sched_yield จาก Linux man-pages บน Ubuntu 24.04.1 (glibc 2.39)
3. man 1 taskset จาก package util-linux และ man 1 objdump จาก package binutils
4. GCC Manual, *Built-in Functions for Memory Model Aware Atomic Operations* (__atomic_fetch_add)

ภาคผนวก: ซอร์สโค้ดทั้งหมดของ ps7.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <pthread.h>
5  #include <sched.h>
6
7  #define INITIAL_BALANCE 1000L
8  #define AMOUNT         1L
9  #define MAX_THREADS    64
10
11 long balance = INITIAL_BALANCE;
12
13 int use_yield = 0;           /* set once in main() before any thread starts */
14
15 enum role { DEPOSITOR, WITHDRAWER, BOTH };
16
17 struct atm {
18     pthread_t tid;
19     enum role role;
20     long tx;                  /* transactions this thread must make */
21     long deposits;           /* written once by the thread when done: */
22     long withdrawals;        /* how many transactions it really made */
23 };
24
25 void deposit_plain(void)
26 {
27     balance = balance + AMOUNT;
28 }
29
30 void withdraw_plain(void)
31 {
32     balance = balance - AMOUNT;
33 }
34
35 void deposit_yield(void)
36 {
37     long reg = balance;      /* Load */
38     reg = reg + AMOUNT;      /* modify */
39     sched_yield();           /* force a context switch */

```

```

40     balance = reg;                /* store */
41 }
42
43 void withdraw_yield(void)
44 {
45     long reg = balance;
46     reg = reg - AMOUNT;
47     sched_yield();
48     balance = reg;
49 }
50
51 void *atm_thread(void *arg)
52 {
53     struct atm *a = arg;
54     void (*deposit)(void) = use_yield ? deposit_yield : deposit_plain;
55     void (*withdraw)(void) = use_yield ? withdraw_yield : withdraw_plain;
56     long done_dep = 0, done_wdr = 0;
57     long i;
58
59     if (a->role == DEPOSITOR || a->role == BOTH)
60         for (i = 0; i < a->tx; i++) {
61             deposit();
62             done_dep++;
63         }
64     if (a->role == WITHDRAWER || a->role == BOTH)
65         for (i = 0; i < a->tx; i++) {
66             withdraw();
67             done_wdr++;
68         }
69
70     a->deposits = done_dep;
71     a->withdrawals = done_wdr;
72     return NULL;
73 }
74
75 static void usage(const char *prog)
76 {
77     fprintf(stderr, "usage: %s [threads] [tx_per_thread] [yield]\n"
78                  "    threads = 1, or an even number from 2 to %d\n"
79                  "    yield = call sched_yield() between load and store\n",
80             prog, MAX_THREADS);
81     exit(1);
82 }
83
84 int main(int argc, char *argv[])
85 {
86     struct atm atms[MAX_THREADS];
87     int nthreads = 1;
88     long tx = 1000000;
89     long deposits = 0, withdrawals = 0, expected;
90     int i;
91
92     if (argc > 1)
93         nthreads = atoi(argv[1]);
94     if (argc > 2)
95         tx = atol(argv[2]);
96     if (argc > 3) {
97         if (strcmp(argv[3], "yield") != 0)
98             usage(argv[0]);
99         use_yield = 1;
100     }
101     if (nthreads < 1 || nthreads > MAX_THREADS
102         || (nthreads > 1 && nthreads % 2 != 0) || tx < 1)
103         usage(argv[0]);
104
105     memset(atms, 0, sizeof atms);
106     for (i = 0; i < nthreads; i++) {
107         atms[i].tx = tx;
108         if (nthreads == 1)
109             atms[i].role = BOTH;
110         else
111             atms[i].role = (i < nthreads / 2) ? DEPOSITOR : WITHDRAWER;
112     }
113
114     /* start every ATM first, then wait for all of them to finish */

```

```
115     for (i = 0; i < nthreads; i++)
116     {
117         if (pthread_create(&atms[i].tid, NULL, atm_thread, &atms[i]) != 0) {
118             perror("pthread_create");
119             return 1;
120         }
121         for (i = 0; i < nthreads; i++)
122             pthread_join(atms[i].tid, NULL);
123
124         for (i = 0; i < nthreads; i++) {
125             deposits    += atms[i].deposits;
126             withdrawals += atms[i].withdrawals;
127         }
128         expected = INITIAL_BALANCE + deposits * AMOUNT - withdrawals * AMOUNT;
129
130         printf("[%s] threads=%d tx=%ld in=%ld out=%ld | expected=%ld actual=%ld diff=%+ld %s\n",
131             use_yield ? "yield" : "plain", nthreads, tx, deposits, withdrawals, expected, balance,
132             balance - expected, (balance == expected) ? "OK" : "RACE");
133     }
134     return 0;
135 }
```

Code 7: ซอร์สโค้ดฉบับสมบูรณ์ PS07/src/ps7.c